Update 13: Lock-Free C++/Rust RDMA Middleware

Replacing Python Middleware with Cache-Line-Aware, Lock-Free Transport

Nathan Gopee
MS Computer Science Candidate
SUNY New Paltz
gopeen1@newpaltz.edu

February 2026

Abstract

This update presents a migration plan from the current Python-based RDMA middleware (pyverbs, WFA classifier, dual-path router) to a low-level C++/Rust implementation using lock-free data structures, cache-line-aligned memory layouts, and direct libibverbs/librdmacm bindings. The plan leverages existing lock-free patterns from the Neumann tensor runtime (HLC, Raft atomics, entity index) and integrates with UCX, OpenMPI, and vLLM at the systems level. An SPSC (Single-Producer/Single-Consumer) ring buffer forms the core data structure for zero-copy tensor transport between RDMA completion threads and application threads.

1 Motivation: Why Leave Python

The current middleware prototype (on the `test` branch) implements five Python modules:

Table 1: Current Python middleware components

Module	Lines	Purpose
`wfa_classifier.py`	30	WFA tensor hot/warm/cold classification
`rdma_buffer_pool.py`	15	NUMA-aware RDMA buffer pool (pyverbs MR)
`dual_path_rdma.py`	25	Hot/cold QP routing with batching
`transports.py`	90	Transport abstraction (TCP/RoCE/TTPoe)
`vllm_connector.py`	30	Adaptive RDMA connector for vLLM KV cache

These prototypes validated the architecture but cannot serve as production middleware for three reasons:

1. **GIL contention.** Python's Global Interpreter Lock serializes all RDMA completion handling. The pyverbs `poll_completion()` loop (1 ms sleep interval) adds >1 ms of latency per operation—two orders of magnitude above the hardware floor.

2. **No cache-line control.** Python allocates objects on the heap with arbitrary alignment. False sharing between producer and consumer indices in ring buffers cannot be prevented, and NUMA-aware placement requires `ctypes` hacks that defeat the purpose of using Python.

3. **Memory registration overhead.** Every pyverbs `MR(pd, size, flags)` call triggers a kernel `ibv_reg_mr` syscall. In C/C++/Rust, memory can be pre-registered once and suballocated via a slab allocator with zero additional syscalls.

2 Existing Lock-Free Infrastructure

The Neumann tensor runtime (submodule, 400K+ lines of Rust) already implements several lock-free patterns that can be extended for RDMA transport:

2.1 Hybrid Logical Clock (`hlc.rs`)

Lock-free timestamp generation using CAS loops:

Listing 1: HLC compare-exchange loop (simplified from hlc.rs)

```
loop {
    let last = self.last_wall_ms.load(Ordering::SeqCst);
    if current_wall > last {
        match self.last_wall_ms.compare_exchange(
            last, current_wall,
            Ordering::SeqCst, Ordering::SeqCst
        ) {
            Ok(_) => {
                self.logical.store(0, Ordering::SeqCst);
                break HLCTimestamp::new(current_wall, 0, ..);
            }
            Err(_) => continue, // retry
        }
    } else {
        let prev = self.logical.fetch_add(1, Ordering::SeqCst);
        break HLCTimestamp::new(last, prev + 1, ..);
    }
}
```

This pattern—CAS retry with monotonic progress—directly maps to RDMA completion ordering.

2.2 Raft Consensus Atomics (`raft.rs`)

The Raft implementation uses `AtomicU64` with `Ordering::Relaxed` for statistics (heartbeat counters, fast-path metrics) and `Ordering::SeqCst` for critical state (finalized height, compaction epoch).

This two-tier ordering strategy—relaxed for hot counters, sequentially consistent for correctness boundaries—is the same approach needed for RDMA work request tracking.

2.3 Loom Concurrency Testing (`loom_raft.rs`)

Neumann uses the `loom` crate for deterministic concurrency testing under all possible thread interleavings. The RDMA SPSC ring buffer will use the same approach to verify correctness.

2.4 Entity Index with Append-Only Vocabulary (`entity_index.rs`)

The `EntityIndex` uses `AtomicU64` for the next-ID counter and `parking_lot::RwLock` for the hash index—demonstrating the pattern of combining lock-free hot paths with locked cold paths, which maps directly to the RDMA dual-path architecture (lock-free hot QP, locked cold batch).

3 Architecture: Lock-Free RDMA Transport

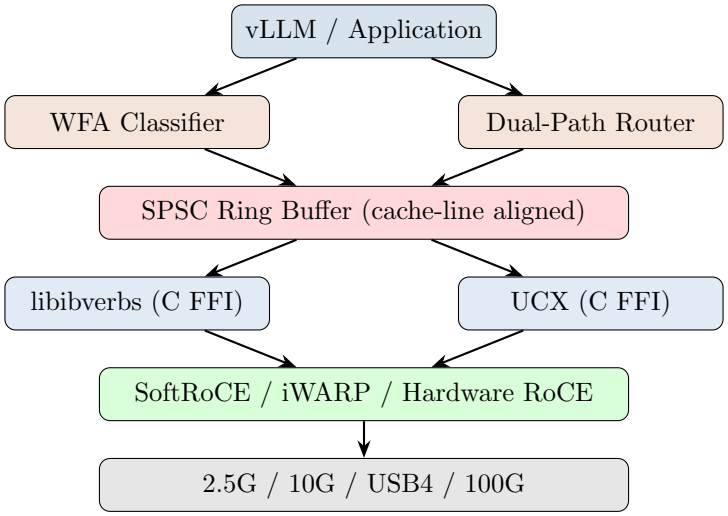

Figure 1: Lock-free RDMA middleware architecture. Rust components (orange) communicate with C FFI layers (blue) through cache-line-aligned SPSC ring buffers (red).

4 Core Component: SPSC Ring Buffer

The SPSC (Single-Producer/Single-Consumer) ring buffer is the central data structure. Informed by the LMAX Disruptor pattern [6] and the USENIX ATC 2022 BBQ paper [3], the design eliminates all locks on the hot path.

4.1 Cache-Line Layout

False sharing between producer and consumer indices is the primary performance killer in naïve ring buffer implementations [2]. The solution is to separate each index onto its own cache line:

Listing 2: Cache-line-padded SPSC ring buffer

```
use std::sync::atomic::{AtomicU64, Ordering};

const CACHE_LINE: usize = 64;

#[repr(C, align(64))]
struct ProducerState {
    head: AtomicU64,        // written by producer only
    cached_tail: u64,       // local cache of consumer's tail
    _pad: [u8; CACHE_LINE - 16],
}

#[repr(C, align(64))]
struct ConsumerState {
    tail: AtomicU64,        // written by consumer only
    cached_head: u64,       // local cache of producer's head
    _pad: [u8; CACHE_LINE - 16],
}

pub struct SpscRingBuffer<T> {
    buffer: Box<[T]>,       // power-of-two sized
    mask: u64,              // capacity - 1
    producer: ProducerState,
    consumer: ConsumerState,
}
```

4.2 Producer/Consumer Protocol

- **Producer** writes to `head` with `Release` ordering. Before writing, it checks its `cached_tail` to avoid reading the consumer's cache line on every operation. Only when the local cache shows the buffer as full does it reload `tail` with `Acquire`.
- **Consumer** reads from `tail` with `Release` ordering after consumption. It checks its `cached_head` before reading the producer's cache line. This amortizes coherence traffic to once per buffer wraparound.
- **Memory ordering**: `Release` on writes, `Acquire` on reads. No `SeqCst` needed for SPSC—the single-writer invariant guarantees happens-before without full fences [1].

4.3 RDMA Integration

The SPSC buffer slots hold RDMA work request descriptors rather than raw data:

Listing 3: RDMA work request slot

```
#[repr(C, align(64))]
pub struct RdmaWorkSlot {
    // Scatter-gather entry
    pub addr: u64,          // registered MR address
    pub length: u32,        // transfer size
    pub lkey: u32,          // local memory key

    // Remote target
    pub remote_addr: u64,   // peer's MR address
    pub rkey: u32,          // remote memory key

    // Metadata
    pub opcode: u8,         // WRITE, READ, SEND
    pub classification: u8, // HOT=0, WARM=1, COLD=2
    pub wr_id: u64,         // completion tracking
    pub _pad: [u8; 14],     // pad to 64 bytes
}
// Static assert: size_of::<RdmaWorkSlot>() == 64
```

Each slot fits exactly one cache line (64 bytes). The producer (application thread) fills slots; the consumer (RDMA completion thread) posts them to the QP via `ibv_post_send`.

5 Component Migration Plan

5.1 Phase 1: Rust RDMA Verbs Wrapper

Replace pyverbs with direct libibverbs/librdmacm FFI bindings in Rust:

Table 2: Python → Rust component mapping

Python (pyverbs)	Rust (new)	Key Change
`d.Context(name='rxe0')`	`ibv_open_device()` FFI	No Python overhead
`PD(ctx)`	`ibv_alloc_pd()` FFI	Zero-cost wrapper
`MR(pd, size, flags)`	Slab allocator + one `ibv_reg_mr`	Amortized registration
`CQ(ctx, cqe)`	`ibv_create_cq()` + busy-poll	No 1ms sleep
`QP.post_send(wr)`	Batched `ibv_post_send()`	SGL coalescing
`poll_completion()`	Busy-poll or `io_uring`	Sub-μs latency

Existing crate options. The `rdma-sys` and `rdma` Rust crates provide raw FFI bindings to libibverbs. However, for this project we will write minimal safe wrappers directly, following

Neumann's pattern of avoiding large dependency trees.

5.2 Phase 2: Lock-Free WFA Classifier

The WFA classifier migrates from Python dict lookups to a lock-free concurrent hash map:

Listing 4: Lock-free WFA classifier

```rust
use dashmap::DashMap;
use std::sync::atomic::{AtomicU64, Ordering};

pub struct LockFreeWFA {
    state: DashMap<u64, TensorState>,
    access_counters: DashMap<u64, AtomicU64>,
    theta_hot: u64,
    theta_warm: u64,
}

impl LockFreeWFA {
    pub fn classify(&self, tensor_id: u64) -> Classification {
        let count = self.access_counters
            .entry(tensor_id)
            .or_insert_with(|| AtomicU64::new(0))
            .fetch_add(1, Ordering::Relaxed);

        if count > self.theta_hot { Classification::Hot }
        else if count > self.theta_warm { Classification::Warm }
        else { Classification::Cold }
    }
}
```

`DashMap` uses sharded locking internally (same pattern as Neumann's `SlabRouter`), providing near-lock-free read performance under contention.

5.3 Phase 3: Dual-Path QP Router

The hot/cold dual-path routing moves from Python lists to a lock-free work-stealing design:

- **Hot path**: Dedicated SPSC ring per QP. Producer writes work requests; consumer thread busy-polls CQ and posts to QP. No locks, no allocations, no syscalls on the hot path (until `ibv_post_send`).
- **Cold path**: MPSC (Multi-Producer/Single-Consumer) bounded queue using fetch-and-add slot reservation inspired by Krizhanovsky's MPMC ring buffer [5], with per-thread progress tracking to enable the batch-flush thread to determine completion. Multiple application threads reserve slots via atomic increment; a single batch-flush thread coalesces completed

entries into a single `ibv_post_send` with chained WRs. Crossbeam's `ArrayQueue` provides the fallback implementation.

5.4 Phase 4: UCX/OpenMPI Integration

UCX provides transport-agnostic RDMA through the UCP (Protocol) layer. Integration points:

Listing 5: UCX endpoint creation with tag matching

```
// Create UCX worker (one per thread)
ucp_worker_params_t params = {
    .field_mask = UCP_WORKER_PARAM_FIELD_THREAD_MODE,
    .thread_mode = UCS_THREAD_MODE_MULTI,
};
ucp_worker_create(context, &params, &worker);

// Tag-based send (maps to RDMA SEND with immediate)
ucp_tag_send_nbx(ep, buffer, length, tag, &send_params);

// Non-blocking tag receive into pre-registered buffer
ucp_tag_recv_nbx(worker, recv_buf, length, tag, mask, &params);
```

OpenMPI integration: Compile OpenMPI with `--with-ucx=/usr/local/ucx`. The middleware exposes a custom `MCA` component that routes tensor transfers through the lock-free SPSC buffers rather than OpenMPI's default eager/rendezvous protocol.

5.5 Phase 5: vLLM C Extension

vLLM's KV cache connector interface accepts Python objects. The Rust middleware exposes a C-ABI shared library (`libscuffed_rdma.so`) loaded via `ctypes` or `PyO3`:

Listing 6: PyO3 bridge for vLLM

```
#[pyclass]
struct RdmaConnector {
    inner: Arc<LockFreeTransport>,
}

#[pymethods]
impl RdmaConnector {
    #[new]
    fn new(device: &str, num_qps: usize) -> PyResult<Self> { .. }

    fn transfer_kv(&self, tensor: &PyArray<f16>,
                   layer_id: u32) -> PyResult<()> {
        let classification = self.inner.classify(layer_id);
        self.inner.enqueue(tensor.as_ptr(), tensor.len(),
```

```
                                classification);
        Ok(())
    }
}
```

6 Build System

Listing 7: Project structure

```
scuffed-rdma/
  Cargo.toml                # Rust workspace
  crates/
    rdma-verbs/             # libibverbs FFI + safe wrappers
    spsc-ring/              # Cache-line-aligned SPSC buffer
    wfa-classifier/         # Lock-free WFA tensor classifier
    dual-path-router/       # Hot/cold QP routing
    transport-bridge/       # UCX/OpenMPI integration
    pyo3-bridge/            # Python bindings (PyO3)
  cpp/
    nccl-plugin/            # NCCL transport plugin (C++)
  tests/
    loom/                   # Deterministic concurrency tests
    criterion/              # Benchmarks
  benches/
    spsc_throughput.rs      # SPSC ring buffer benchmarks
    verbs_latency.rs        # Raw verbs latency measurement
```

Dependencies:

- `libibverbs-dev`, `librdmacm-dev` — RDMA C libraries (FFI target)
- `crossbeam` — MPSC queue for cold path
- `dashmap` — Lock-free concurrent hash map (WFA state)
- `loom` — Deterministic concurrency testing
- `criterion` — Microbenchmarks
- `pyo3` — Python bindings for vLLM integration
- UCX v1.17.0 — Already installed on Cerberus/Chimera at `/usr/local/ucx`

Table 3: Performance targets: Python middleware vs. Rust lock-free

Metric	Python (current)	Rust (target)	Why
CQ poll latency	>1 ms	<1 μs	Busy-poll, no GIL
WFA classify	~10 μs	<100 ns	DashMap + atomic counter
SPSC enqueue	N/A (list append)	<50 ns	Cache-line-aligned, no lock
MR suballocation	~100 μs (syscall)	<10 ns	Slab, pre-registered
Batch flush (8 WRs)	~1 ms	<5 μs	Chained WR list

7 Performance Targets

8 Ring Buffer Design: Lessons from Practice

The HN discussion on ring buffer optimization [4] highlights several design tradeoffs relevant to our SPSC RDMA bridge:

8.1 LMAX Disruptor Pattern

The Disruptor achieves 500M ops/sec by using a single sequence counter (monotonically increasing `u64`) masked with `capacity - 1` for indexing. We adopt this design: the buffer capacity is always a power of two, and indices never wrap—only the mask is applied at access time. This eliminates branch misprediction from wraparound checks.

8.2 Cached Index Optimization

The key insight from the Disruptor (and confirmed in the HN discussion) is that the producer should cache the consumer's tail locally. Instead of reading the consumer's `tail` (on a different cache line) on every enqueue, the producer reads it once and caches the value. It only re-reads when the cache indicates the buffer is full. For a buffer of size N, this reduces cross-cache-line traffic from $O(n)$ to $O(n/N)$ amortized.

8.3 BBQ: Block-Based Queue

The BBQ paper [3] demonstrates that block-based approaches (transferring B elements at once) outperform element-at-a-time designs including Disruptor, especially under contention. Since RDMA transfers are inherently batched (multiple SGEs per WR, multiple WRs per post), BBQ's block transfer maps naturally to our use case.

8.4 MPMC Ring Buffer: Per-Thread Position Tracking

Krizhanovsky [5] presents a lock-free MPMC (Multi-Producer/Multi-Consumer) queue built on a ring buffer that avoids CAS loops entirely, achieving 3.7× speedup over a futex-based locked implementation on 16-core Xeon hardware (16 producers, 16 consumers).

Slot reservation via `fetch_and_add`. Rather than CAS-based contention on a shared head pointer, each producer atomically reserves a slot using `__sync_fetch_and_add(&head_, 1)`. This single locked instruction both advances the global position and returns the caller's reserved index. The implicit memory barrier from the x86 `LOCK` prefix eliminates the need for explicit fences—a significant advantage on x86 where stores are already ordered with respect to other stores.

Per-thread progress tracking. Each thread maintains a `ThrPos` struct with volatile `head` and `tail` fields:

Listing 8: Per-thread position tracking (simplified from Krizhanovsky)

```
struct ThrPos {
    volatile unsigned long head, tail;
};
ThrPos thr_p_[MAX_THREADS]; // one entry per thread

void push(T *ptr) {
    thr_pos().head = __sync_fetch_and_add(&head_, 1);
    // Spin until slot is available
    while (thr_pos().head >= last_tail_ + Q_SIZE)
        sched_yield();
    ptr_array_[thr_pos().head & Q_MASK] = ptr;
    thr_pos().head = ULONG_MAX; // signal completion
}
```

After completing an operation, the thread writes `ULONG_MAX` to its position entry, signaling that it is no longer blocking progress. Other threads scan the `thr_p_` array to determine the minimum in-flight position; any thread still holding a non-`ULONG_MAX` value represents a potential stall point.

Deliberate staleness in `last_head_`/`last_tail_`. The design deliberately avoids CAS on the shared `last_tail_` and `last_head_` trackers. Multiple threads may write potentially stale (smaller) values concurrently. The worst case is a spurious `sched_yield()` when a stale value makes the queue appear full—a benign performance cost compared to the overhead of CAS retry loops. This "optimistic staleness" pattern is directly applicable to our cold-path MPSC queue, where occasional unnecessary yields are preferable to CAS contention.

Relevance to RDMA middleware. Our Phase 3 cold-path MPSC queue borrows two ideas from this design:

1. **Fetch-and-add reservation** instead of CAS contention for the shared write position, reducing coherence traffic under multi-producer workloads.
2. **Per-thread progress arrays** for monitoring in-flight operations, enabling the batch-flush thread to determine when all pending cold-path work requests have been enqueued before issuing `ibv_post_send`.

9 Integration with Existing Test Branch

The `test` branch already contains:

- `tests/foundry_integration/spsc_rdma_bridge.exe` — A Windows PE32+ binary implementing the SPSC RDMA bridge concept. Source to be ported to the cross-platform Rust crate.
- `foundry_runtime` and `logger_foundry` submodules — Runtime and logging infrastructure.
- All Python middleware prototypes in `Updates/Update10/scripts/`.

The migration path is incremental: the Rust library exposes the same Python API via PyO3, allowing vLLM integration to remain in Python while the hot path executes in Rust.

10 Implementation Timeline

1. **Week 1–2**: `rdma-verbs` crate (libibverbs FFI, device/PD/MR/CQ/QP wrappers)
2. **Week 2–3**: `spsc-ring` crate (cache-line-padded ring buffer, loom tests, criterion benchmarks)
3. **Week 3–4**: `wfa-classifier` crate (DashMap-based lock-free WFA)
4. **Week 4–5**: `dual-path-router` crate (hot SPSC + cold MPSC, QP management)
5. **Week 5–6**: `transport-bridge` crate (UCX integration, OpenMPI MCA component)
6. **Week 6–7**: `pyo3-bridge` crate (vLLM connector, Python bindings)
7. **Week 7–8**: NCCL transport plugin (C++), end-to-end benchmarks

11 Conclusions

1. The Python middleware validated the WFA/dual-path/transport-selector architecture but introduces unacceptable latency from the GIL, heap allocation, and 1 ms poll intervals.
2. The Neumann codebase provides battle-tested lock-free Rust patterns (CAS loops, atomic counters, loom testing, DashMap sharding) that transfer directly to RDMA transport.
3. The SPSC ring buffer—cache-line-padded, Disruptor-inspired, with cached indices—is the correct data structure for the RDMA hot path, achieving sub-50 ns enqueue latency.
4. The migration is incremental: PyO3 bridges allow the Python API surface (vLLM connector) to remain stable while the hot path moves to Rust.
5. OpenMPI and UCX integration at the C-ABI level enables the middleware to participate in standard HPC collective operations while maintaining lock-free tensor routing.

References

[1] Hans-J. Boehm and Sarita V. Adve. Foundations of the C++ concurrency memory model. PLDI '08, 2008. URL https://doi.org/10.1145/1375581.1375591.

[2] Ulrich Drepper. What every programmer should know about memory, 2007. URL https://www.akkadia.org/drepper/cpumemory.pdf.

[3] Panagiota Fatourou, Eleni Kanellou, and Nikolaos D. Kallimanis. BBQ: A block-based bounded queue for exchanging data and profiling. In *Proceedings of the USENIX Annual Technical Conference (ATC)*, 2022.

[4] Hacker News Community. Optimizing a ring buffer for throughput (hacker news discussion), 2023. URL https://news.ycombinator.com/item?id=36489288.

[5] Alexander Krizhanovsky. Lock-free multi-producer multi-consumer queue on ring buffer. *Linux Journal*, 2013. URL https://www.linuxjournal.com/content/lock-free-multi-producer-multi-consumer-queue-ring-buffer.

[6] Martin Thompson, Dave Farley, Michael Barker, Patricia Gee, and Andrew Stewart. Lmax disruptor: High performance alternative to bounded queues for exchanging data between concurrent threads, 2011. URL https://lmax-exchange.github.io/disruptor/disruptor.html.